

Arnavutköy Belediyesi Akbi Simülasyon Sistemi

Toplu ulaşım kartının yaşam döngüsünü uçtan uca modelleyen
web, mobil ve sunucu bileşenlerinden oluşan bir simülasyon platformu

Hazırlayanlar	Ömer Faruk Türkdoğan Kaan Üstüntaş
Staj Kurumu	Arnavutköy Belediyesi
Staj Sorumlusu	Özcan Fatih Can
Proje Adı	AkBiSis
Dönem	Staj — 1. Hafta
Rapor Tarihi	Temmuz 2026
Proje Deposu	github.com/OFThub/AkBiSis

1. Özet

Bu rapor, staj kapsamında geliştirilen **Arnavutköy Belediyesi Akbil Simülasyon Sistemi** projesini ve bu proje süresince yürütülen yazılım geliştirme çalışmalarını belgelemektedir. Proje; bir toplu ulaşım kartının (akbil) günlük kullanımını —otobüse binme, inme, yolculuk kaydı ve hat yoğunluğu analizi— gerçek bir donanıma ihtiyaç duymadan uçtan uca simüle eden üç bileşenli bir yazılım sistemidir.

Sistem; Python/FastAPI tabanlı bir **sunucu (backend)**, saf HTML/CSS/JavaScript ile geliştirilen bir **web arayüzü** ve Expo/React Native ile yazılmış bir **mobil uygulamadan** oluşur. Tüm iş kuralları sunucuda uygulanır; istemciler yalnızca veriyi gösterir. Proje toplamda yaklaşık **8.300 satır kaynak kod**, **39 API uç noktası**, **8 veritabanı tablosu** ve dört ayrı teknik dökümandan meydana gelmektedir.

2. Giriş ve Amaç

Staj sürecinin bu aşamasında amaç, modern bir yazılım projesinin tüm katmanlarını —veritabanı tasarımından mobil arayüze kadar— birlikte ele alarak gerçek bir mühendislik problemine tam kapsamlı bir çözüm üretmektir. Seçilen problem, bir belediyenin toplu ulaşım sistemidir.

Gerçek bir akbil sisteminde kart okuyucu donanımı, fiziksel otobüsler ve saha altyapısı gerekir. Bu projede söz konusu unsurların tamamı **yazılımsal olarak simüle edilmiştir**: Otobüsler sanaldır ve gerçek saate göre hatları üzerinde hareket eder; yolcu, telefonundan “kart basarak” otobüse biner ve iner; belediye ise bu yolculuklardan hat yoğunluğu ve sefer ihtiyacı analizleri çıkarır.

Projenin bilinçli bir tasarım kararı olarak, sistemde **ücret ve bakiye kavramı bulunmaz**. “Tam”, “Öğrenci” ve “65+” yalnızca bir yolcu statüsüdür; amaç para akışını değil, hareketlilik ve yoğunluk verisini modellemektir.

3. Proje Genel Bakışı

Sistem, herbiri ayrı bir kullanıcı kitlesine hizmet eden üç bileşenden oluşur:

Bileşen	Kullanıcı	İşlevi
Sunucu (backend)	—	Kimlik doğrulama, otobüs simülasyonu, yolculuk kayıtları ve yönetim analitiği. Web sitesini de bu süreç sunar.
Web arayüzü	Yolcu / Yönetici	Giriş-kayıt, sanal kart görünümü, hat yoğunlukları ve (yöneticiler için) analiz paneli.
Mobil uygulama	Yolcu	Otobüse binme/inme (kart basma), kart bilgileri, favori hatlar ve yolculuk geçmişi.

Üç bileşen tek bir hesap sistemini paylaşır: mobil uygulamada yapılan bir yolculuk, aynı hesapla girilen web panelinde anında görüntülenebilir.

4. Kullanılan Teknolojiler

Proje boyunca güncel ve yaygın kullanılan teknolojiler tercih edilmiştir:

Katman	Teknoloji	Kullanım Amacı
Sunucu dili	Python 3.12+	Backend uygulama dili
Web çatısı	FastAPI	REST API ve otomatik OpenAPI dökümanı
Veritabanı	PostgreSQL 16	Kalıcı veri saklama (Docker ile)
ORM & Göç	SQLAlchemy 2.0 · Alembic	Nesne-ilişki eşleme ve şema göçleri
Doğrulama	Pydantic v2	İstek/yanıt şema doğrulaması
Güvenlik	JWT (python-jose) · bcrypt	Token tabanlı kimlik ve parola karma
Web arayüzü	HTML · CSS · JavaScript	Derleme adımı olmayan saf ön yüz
Grafikler	Chart.js	Yoğunluk ve trend grafikleri
Mobil	Expo SDK 57 · React Native 0.86 TypeScript (strict)	Çapraz platform mobil uygulama
Mobil dil	Docker · Git	Tip güvenli mobil geliştirme
Araçlar		Ortam yönetimi ve sürüm kontrolü

5. Sistem Mimarisi

Sunucu,katmanlı(layered) bir mimariyle tasarlanmıştır. Her katmanın tek bir sorumluluğu vardır ve bir üst katman yalnızca bir alt katmanı çağırır:

Katman	Sorumluluğu
routes.py	HTTP sözleşmesi: uç noktalar, istek/yanıt şemaları. İş kuralı içermez.
services.py	İş kuralları, doğrulama ve veritabanı işlem (transaction) sınırı.
repositories.py	Tüm SQL sorguları burada toplanır; iş kuralı içermez.
models.py	Veritabanı tablolarının nesne karşılıkları (ORM).
simulation.py	Otobüs konumunu hesaplayan saf matematik; veritabanına dokunmaz.

Tek doğruluk kaynağı ilkesi

Projenin en temel tasarım kararı, **tüm gerçeğin sunucuda yaşamasıdır**. Hat, durak, otobüs konumu, yolculuk ve favori verisi yalnızca sunucuda tutulur; istemciler bunları hesaplamaz veya üretmez, yalnızca görüntüler. Bu kararın somut sonuçları şunlardır:

- Araç konumu sunucuda hesaplandığından, telefonun saati değiştirilse dahi konum kaymaz.
- Biniş durağını sunucu belirler; istemci “şu durakta bindim” diyerek sistemi kandıramaz.
- Yolculuk kaydı iniş anında sunucuda yazılır; sahte kayıt oluşturulamaz.
- Yoğunluk renk kararı sunucuda verilir; iki farklı istemci aynı veriyi farklı yorumlayamaz.

Bir isteğin yaşam döngüsü

Örneğin bir “kart basma” isteği şu yolu izler: Mobil uygulama isteği gönderir → **routes** katmanı kimliği doğrular → **services** katmanı iş kurallarını işletir → **simulation** aracın o anki konumunu hesaplar → **repositories** veritabanına yazar → sonuç istemciye döner. Böylece her katman yalnızca kendi işini yapar ve kod test edilebilir, bakımı kolay bir yapıya kavuşur.

6. Gerçekleştirilen İşler

Proje kapsamında aşağıdaki işlevler baştan sona tasarlanmış ve kodlanmıştır.

6.1 Kimlik doğrulama ve oturum yönetimi

Kullanıcıların kayıt olması ve giriş yapması sağlanmıştır. Parolalar bcrypt ile güvenli biçimde saklanır. Oturumlar JWT ile yürütülür: mobil uygulama token’ı başlıkta, web ise güvenli (httpOnly) çerezde taşır. Aynı doğrulama mantığı tek bir noktada toplanmıştır. Ayrıca “token sürümü” yöntemiyle kullanılmış veya çalınmış bir yenileme (refresh) token’ının iptal edilebilmesi sağlanmıştır.

6.2 Otobüs hareket simülasyonu

Otobüslerin hatlar üzerindeki konumu, herhangi bir veri saklamadan doğrudan **gerçek saatten deterministik olarak** hesaplanır. Her hatta üç otobüs bulunur ve bunlar sefer döngüsüne eşit aralıklarla dağıtılır. Araç her durakta kısa süre bekler, son durakta molaya çekilir ve ters yönde geri döner. Sunucu yeniden başlasa bile aynı anda aynı sonuç üretilir.

6.3 Biniş / iniş (kart basma) mantığı

Sistemin kalbi olanbu işlevde, biniş ve iniş tek bir uç nokta üzerinden yapılır; hangi işlemin gerçekleştiğine sunucu karar verir. Yolcu durağı seçmez — kayda, aracın o an beklediği durak yazılır. Yalnızca durakta bekleyen araca binilebilir; araç yoldayken veya moladayken istek reddedilir. Bir yolcunun aynı anda yalnızca bir açık yolculuğu olabilir.

6.4 Yolculuk kaydı, geçmiş ve otomatik iniş

Her yolculuk veritabanına kaydedilir ve kullanıcı geçmişinde listelenir. Yolcu inmeyi unutursa, otobüs son durağa vardığında sistem tarafından **otomatik olarak indirilir**. Bu iniş anı, yolcu bindiği anda hesaplanıp saklandığından, uygulama kapalı olsa bile doğru çalışır.

6.5 Favori hatlar

Kullanıcı, sıklıkla kullandığı hatları favorilere ekleyebilir. Değişiklikler anında sunucuya yazılır ve hem web hem mobil arayüzde tutarlı biçimde görüntülenir.

6.6 Yönetim paneli ve karar destekli analitik

Belediye yöneticileri için, gerçek yolculuk kayıtlarından beslenen bir analiz paneli geliştirilmiştir. Panel; saatlik yoğunluk, günlük talep trendi, hat bazında doluluk, en yoğun duraklar, kart tipi dağılımı ve en yoğun güzergâhları gösterir. En önemlisi, sistem “bu hatta sefer artırılmalı / azaltılabilir” biçiminde **somut öneriler** üretir. Bu öneri, zirve saatte araç başına düşen yolcu sayısının araç kapasitesine oranından hesaplanır.

6.7 Mobil uygulama arayüzü

Dört sekmeli(Yolculuk,Kart,Geçmiş, Ayarlar) bir mobil uygulama geliştirilmiştir. Hat şeması üzerinde otobüsler canlı olarak izlenebilir; ekran iki saniyede bir kendini günceller. Uygulamada açık/koyu tema ve Türkçe/İngilizce dil desteği vardır. Bu tercihler yalnızca telefonda saklanır.

6.8 Web arayüzü

Derleme adımı gerektirmeyen, saf HTML/CSS/JavaScript ile bir yolcu paneli ve giriş ekranı hazırlanmıştır. Sanal akbil kartı, aktif yolculuk bilgisi, hat yoğunluk grafikleri ve yolculuk geçmişi bu panelde sunulur.

6.9 Veritabanı tasarımı ve göçler

Sekiz tablodan oluşan ilişkisel bir şema tasarlanmış ve Alembic ile beş aşamalı göç (migration) olarak sürümlenmiştir. Veri bütünlüğü, kart başına tek açık yolculuğu garanti eden kısmi tekil indeks gibi veritabanı düzeyi kısıtlarla korunmaktadır.

6.10 Dökümantasyon

Proje; kullanıcı kılavuzu, kurulum/senaryo rehberi ve satır düzeyinde ayrıntılı API-mimari dökümanı olmak üzere üç kapsamlı teknik belge ile ayrıca depo tanıtım dosyası (README) içerecek biçimde belgelenmiştir.

7. Öne Çıkan Teknik Kararlar

Proje boyunca alınan bazı mühendislik kararları, hem sistemin sağlamlığını hem de öğrenme değerini artırmıştır:

Karar	Getirdiği fayda
Durum tutmayan simülasyon	Araç konumu saatten hesaplandığı için sunucu ölçeklenebilir kalır ve yeniden başlatmadan etkilenmez.
Kuralların sunucuda zorlanması	İstemci değiştirilerek kurallar aşılamaz; güvenlik istemciye bırakılmaz.
Kart tipinin anlık damgalanması	Kart tipi sonradan değişse bile geçmiş yolculuklar ve analizler bozulmaz.
Kısmi tekil indeks	Eşzamanlı iki biniş isteği bile “kart başına tek açık yolculuk” kuralını aşamaz.
Token sürümü ile iptal	Çalınan bir yenileme token’ı bir sonraki meşru kullanımla geçersiz kılınır.
Üç katmanlı yönetici koruması	Yönetim sayfası, API ve arayüz ayrı ayrı korunur; adres elle yazılarak erişilemez.

8. Karşılaşılan Zorluklar ve Çözümler

Zorluk	Uygulanan çözüm
Mobilin sunucu adresini bulması	Telefonda “localhost” telefonun kendisini gösterir. Adres, uygulamanın indirildiği geliştirme sunucusunun adresinden otomatik türetilerek elle yapılandırma ihtiyacı ortadan kaldırıldı.
Expo SDK sürüm uyumsuzluğu	Mağazadaki Expo Go istemcisi eski sürümde kaldığında uygulama açılmıyordu; SDK 57 uyumlu istemci ile çözüldü ve dökümana uyarı eklendi.
Eşzamanlı biniş yarış durumu	İki isteğin aynı anda gelmesi uygulama katmanında engellenemediğinden, kural veritabanı düzeyinde tekil indeks ile garanti altına alındı.
Analitikte saat dilimi kayması	Zaman damgaları UTC saklanır; “en yoğun saat” hesabı yerel saat dilimine (Europe/Istanbul) çevrilerek gruplandı.
Kütüphane uyumluluğu (bcrypt)	Yeni Python sürümünde kaldırılan bir modüle bağımlı olan passlib yerine, doğrudan bcrypt API’si kullanıldı.

9. Proje İstatistikleri

Ölçüt	Değer
Toplam kaynak kod (yaklaşık)	~8.300 satır
API uç noktası sayısı	39 (37 API + 2 sistem)

Ölçüt	Değer
Veritabanı tablosu	8
Şema göçü (Alembic)	5
Tanımlı hat / durak	4 hat, 24 benzersiz durak
Simüle edilen otobüs	12 (hat başına 3)
Teknik döküman	4 (kılavuz, senaryo, API-mimari, README)
Desteklenen dil	Türkçe ve İngilizce (mobil)

10. Kazanımlar

Bu proje aracılığıyla,tekbir uygulama üzerinden yazılım geliştirmenin pek çok katmanı bütünsel olarak deneyimlenmiştir:

- Katmanlı mimari kurma ve sorumlulukları ayrıştırma pratiği.
- REST API tasarımı, şema doğrulama ve otomatik dökümantasyon (FastAPI/OpenAPI).
- İlişkisel veritabanı tasarımı, ORM kullanımı ve göçlerle şema sürümleme.
- JWT tabanlı kimlik doğrulama ve pratik güvenlik önlemleri.
- Çapraz platform mobil geliştirme (React Native / TypeScript) ve durum yönetimi.
- Eşzamanlılık, yarış durumu ve veri bütünlüğü konularında çözüm üretme.
- Docker ile geliştirme ortamı yönetimi ve Git ile sürüm kontrolü.
- Farklı okuyucu kitlelerine yönelik profesyonel teknik dökümantasyon yazımı.

11. Sonuç

Staj kapsamında geliştirilen Arnavutköy Akbil Simülasyon Sistemi, gerçek bir toplu ulaşım senaryosunu; sunucu, web ve mobil olmak üzere üç katmanda tutarlı ve güvenli bir biçimde modelleyen tam kapsamlı bir yazılım projesidir. Proje, yalnızca çalışan bir ürün ortaya koymakla kalmamış; aynı zamanda mimari kararların gerekçelerini ve bilinen kısıtlarını açıkça belgeleyen bir dökümantasyon setiyle de desteklenmiştir.

Elde edilen deneyim, modern bir yazılım sisteminin uçtan uca nasıl tasarlanıp hayata geçirildiğine dair bütüncül bir bakış kazandırmış ve sonraki geliştirme aşamaları için sağlam bir temel oluşturmuştur.